

Introduction to Deodesy

Deodesy is designed to provide .NET developers with a reliable and efficient solution for geographical calculations. Originating as a C#/.NET port of Chris Veness's highly respected JavaScript Geodesy Functions, Deodesy aims to maintain the accuracy and breadth of the original library while integrating seamlessly into the .NET environment.

Why Deodesy?

Geographical calculations are fundamental to many applications, from mapping and navigation to logistics and environmental science. Deodesy addresses the need for a robust, well-tested, and comprehensive geodesy library in C#. By porting a widely-used and validated JavaScript library, Deodesy offers:

- **Proven Accuracy:** Leveraging the algorithms and extensive testing of the original Geodesy Functions.
- **C#/.NET Integration:** Native C# implementation, allowing for easy integration into .NET projects without interoperability overhead.
- **Comprehensive Features:** A rich set of functions covering various aspects of geodesic computations.
- **Maintainability:** Written with modern C# practices, ensuring readability and ease of maintenance.

Core Concepts

Deodesy operates on fundamental geodesic principles. Key concepts you'll encounter include:

- **Coordinates:** Representing points on the Earth's surface using latitude and longitude.
- **Distances:** Calculating the shortest path between two points (great-circle distance).
- **Bearings:** Determining the direction from one point to another.
- **Ellipsoidal vs. Spherical Models:** Understanding the differences between modeling the Earth as a perfect sphere versus a more accurate ellipsoid.

Further Reading

- [Getting Started](#)
- [API Reference](#)
- [Original JavaScript Geodesy Functions by Chris Veness](#) 

Namespace Deodesy.Library

Classes

[Coordinate](#)

Represents a point on the Earth's surface using Latitude and Longitude.

[Dms](#)

Provides methods for handling Degrees, Minutes, Seconds (DMS) conversions and formatting.

[LatLonSpherical](#)

Provides functions for spherical geodesy, based on the Haversine formula and other spherical geometry calculations. These functions are adapted from Chris Veness's original JavaScript geodesy library.